

Five things I missed

JAEWON KIM · BILLKIMALT COLUMBIA MARKET SEPTEMBER 2026

First time using Claude Code. For eight years I have done hardware product management, where my job started where the building stopped — yield, RMA rates, field failures. I had never been inside an application while it was being made. None of the five below is bad code; the agent rarely wrote any. They are cases of it being wrong about **context**, and of me being *slow to notice*.

1 I asked for code before we had a spec, and got it

Roughly 2,600 lines across two prototypes, both discarded within days. **HOW I NOTICED** — a teammate pushed a build spec and what I had just commissioned stopped being relevant; days later a second spec landed and it happened again. **WHAT WENT WRONG** — the agent asked which stack to use and I answered, when the honest answer was **“the team has not decided.”** An agent fills a decision vacuum without noticing it is one. In hardware I would never approve tooling before design freeze; here code *felt* free. **WHAT I DID** — closed the pull request, abandoned the branch, and started nothing not traceable to a merged spec.

2 My bug report contained my diagnosis, and my diagnosis was wrong

I reported that the distance filter did nothing, and added that I thought it was not calculating the ZIP code. **HOW I NOTICED** — the agent queried the backend directly: 355 items at 0.5 miles, 1,019 at 10, correct at every setting. **WHAT WENT WRONG** — I bundled a symptom with a cause. The grid did look identical at every radius, because it always loads 24 cards; the cause I supplied was fiction, and accepting it would have cost the session. **WHAT I DID** — report what I saw, and leave the *why* open.

3 The agent’s own tests failed twice, and both failures were in the tests

Its scripts twice reported sign-in broken. Both times the application was fine: one truncated a token out of a quoted-printable email, the other searched the body for a word that only appears in the subject line. **HOW I NOTICED** — the failures were too tidy; everything upstream passed and only the last assertion broke. **WHAT WENT WRONG** — I was reading green or red as a fact about the product. It is a fact about **a harness the agent wrote minutes earlier that nobody reviewed.** **WHAT I DID** — ask what a check asserts before believing it. It paid off when the agent nearly reported “the feed renders more items than exist” — its selector counted every card twice.

4 “Close all the ports” killed a different project

I asked the agent to close ports left from previous sessions. It stopped five processes; two belonged to a different course project. **HOW I NOTICED** — it told me afterwards, named the project, and handed me the restart command. **WHAT WENT WRONG** — my instruction was **a category, not a target**, and broader than I meant. The agent identified every process first, then did what I literally said. One port is still unusable, with dead processes holding the socket. **WHAT I DID** — scope destructive instructions to a target: “stop this project’s servers,” never “close all ports.”

5 Being kept unblocked hid that the repo was broken for everyone else

`requirements.txt` will not install on current Python; three packages fail to build. The agent installed unpinned versions and we carried on. **HOW I NOTICED** — it kept reappearing at the bottom of status reports, and it eventually landed that the app ran on my machine in a state no teammate could reproduce. **WHAT WENT WRONG** — **“keep me moving” and “tell me the repo is broken” are different goals.** The agent did both, but the workaround was instant and the warning was a closing bullet. Speed made the problem quieter. **WHAT I DID** — promoted it to a standing item rather than a footnote. Still unfixed, deliberately: pins affect everyone’s environment.

The through-line is that my detours were not the agent’s errors but **my own, executed faster than I could catch them.** Coming from a field where I inherit finished things and measure how they fail, the surprise was how much of building is deciding what *not* to build yet — and that an agent removes almost every pause where that decision used to happen.